

Covert Channels

2026-09-30

Table of contents

1	Storage Covert Channels	2
2	Timing Covert Channels	3
3	Network Covert Channels	3
4	What's Next	4

A **Covert Channel** in cybersecurity is a method used to secretly transfer information in a way that **violates security policies**. These channels operate by exploiting legitimate communication or computing mechanisms in unintended ways, allowing unauthorized data exfiltration without detection.

A covert channel typically implies the existence of an **overt channel** — a legitimate, sanctioned communication channel being exploited to carry the hidden message. Examples of overt channels include websites, FTP servers, phone conversations, chat applications, social media feeds, and TCP/IP packets.

! Challenge Yourself

Can you think of ways that the above overt channels can be used to communicate covertly? Consider code words, preset conversation systems, puzzles, number ciphers, or steganographic embedding. The more mundane the channel, the harder it is to detect the hidden message.

Because typical covert channels allow communication between entities that are *supposed* to be able to talk to each other — as permitted by some access control policy — they tend to be:

- **Hidden** — they blend into normal, legitimate-looking traffic or system activity
- **Hard to detect** — without knowing what to look for, nothing appears suspicious
- **Hard to set up** — they may require administrative access to machines or services
- **Low bandwidth** — they require a great deal of time or computational resources to transmit even a small amount of data

That last point is worth dwelling on. Sending a single ASCII character over a covert channel may require many overt messages or significant timing overhead. Covert channels trade efficiency for secrecy.

i Note

Steganography (hiding data *within* data, e.g. embedding text inside an image) is distinct from a covert channel. The key difference is that steganography requires a pre-existing channel — you must already be able to send a file or message to your recipient. Covert channels, by contrast, *create* a channel from scratch by exploiting the unintended side effects of a system's normal operation. Both are clandestine; they achieve it differently.

Covert channels fall into two main categories:

Category	Mechanism
Storage	Sender modifies a shared resource; receiver reads it
Timing	Sender varies system timing or resource availability; receiver measures it

1 Storage Covert Channels

With this category, communication takes place by subtly modifying a **shared storage location**. The sender (or sending process) modifies a general resource that everyone has access to, and the receiver reads that resource to extract the hidden message. No direct communication between sender and receiver is needed.

Here are three concrete examples:

1. Abusing the print queue. The print queue keeps track of what jobs a printer has to work on, and everyone has access to it. A sender could submit jobs to the queue to signal a 1, or deliberately skip submissions to signal a 0. A receiver polling the queue at regular intervals can reconstruct the bitstream and decode the message over time.

2. Abusing web server log files. Web servers record every page request. A sender could access a designated page to transmit a 0, and a different page to transmit a 1. A receiver with read access to the log file can parse the access pattern and reconstruct the hidden message — with no suspicious direct communication between parties.

3. FTP file permissions. FTP servers are easy to deploy and loosely supervised. A sender can embed a message in the permission bits of files on the server. Casual users see only a file listing; a receiver who knows the encoding scheme can read the hidden message from the permission bits. Each file's 10-character permission string can carry up to **10 bits** of information:

```
drwxrwxrwx 1 prof prof 0 Mar 26 17:52 'Some random file name'
```

With the simple scheme — *letter present = 1, dash = 0* — each file contributes up to 10 bits to the covert message.

Note

FTP file permissions are the basis of the **Storage Covert Channel Lab** that accompanies these notes. By the end of that lab you will be able to embed and decode a text message using only the permission bits of files on an FTP server.

2 Timing Covert Channels

This category relies on **measurable variations in system timing or resource availability** to transmit hidden messages. The receiver must watch the system and measure intervals — it never reads a file or accesses a log directly.

1. File access duration. A sender opens and closes a file for varying durations: a short hold signals 0, a long hold signals 1. The receiver, watching the file lock or access time, accumulates bits and decodes the message. This can work entirely within a single machine.

2. Port knocking patterns. Port knocking (covered later in the course) is a technique for keeping network ports hidden until a secret knock sequence is recognised. A sender can deliberately use an *incorrect* knock pattern to encode bits. A receiver inside the network observes the pattern and decodes the message — even though the ports remain closed and traffic appears normal from the outside.

3. Hard drive acoustic side channel. Mechanical hard drives produce distinct sounds based on how the drive head is moving. Researchers demonstrated that these sounds can be deliberately controlled by a process to exfiltrate data from an *air-gapped* computer — one completely disconnected from the internet. A nearby device with a microphone records and decodes the audio.¹

Note

The **Timing Covert Channel Lab** will have you build a TCP server that transmits an overt text message character-by-character, with the delay between each character encoding a hidden binary message. Short delay = 0; long delay = 1.

3 Network Covert Channels

Strictly speaking this is not a separate fundamental category — any example here could be placed into either of the two categories above. However, these attacks share a common feature: the exploitation of **network infrastructure** as the carrier.

A classic example is embedding a covert message in the **header fields of network packets**. The overt payload (the body of the packet) carries legitimate-looking data, and many security systems

¹Guri et al., “DiskFiltration: Data Exfiltration from Speakerless Air-Gapped Computers via Covert Hard Drive Noise.” arXiv:1608.03431. <https://arxiv.org/abs/1608.03431>

inspect the payload for suspicious content. However, certain header fields — IP identification numbers, TCP sequence numbers, TTL values, reserved bits — receive far less scrutiny. A sender places bits of the covert message in these fields; a receiver who knows where to look reconstructs the hidden message packet by packet as traffic flows through the network.

Because normal traffic is the cover story, these channels are extremely difficult to distinguish from legitimate network activity without deep packet inspection policies specifically designed to detect covert encoding.

4 What's Next

Note

You will now work through **two labs** in which you will set up one covert channel from each of the two main categories described above. By the end of the labs, you will be able to both send and receive messages using either channel.

The discussion in this chapter and the labs that follow are not intended to be a comprehensive survey of all forms covert channels might take. Rather, they are designed to expose you to specific forms that have been used in the past. Once you see one, you should start to recognise opportunities everywhere: any shared resource, any observable timing difference, any underspecified header field is a potential channel.

For a rigorous treatment of this topic — including formal models, detection theory, and access control logic — enroll in **CSC 446 / CYEN 402: Access Control Logic and Covert Channels**, offered every other year (last offered Winter 2024–25).